

Exercise 13

AIM :

To Write a C program for the below requirements

1. Create a binary tree with create(), insert(), delete() and search() functions

ALGORITHM :

Step 1: Start the Program

1. Begin execution of the program.

Step 2: Define the Tree Node Structure

2. Define a structure TreeNode containing:
 - Integer data
 - Pointer left to the left child
 - Pointer right to the right child

Step 3: Declare Required Functions

3. Declare functions for:
 - Creating a node
 - Inserting a node
 - Deleting a node
 - Searching a node
 - Finding minimum and maximum values
 - Tree traversals (inorder, preorder, postorder)
 - Calculating height and node count
 - Displaying the tree
 - Freeing allocated memory

Step 4: Initialize Root

4. Initialize the root pointer root to NULL to represent an empty tree.

Step 5: Display Main Menu in a Loop

5. Repeat the following steps until the user selects Exit:

- Display menu options:
 1. Create/Insert Node
 2. Delete Node
 3. Search Node
 4. Display Tree
 5. Tree Traversals
 6. Tree Statistics
 7. Exit
- Read the user's choice.

Step 6: Perform Operations Based on Choice

Case 1: Create / Insert Node

6. Read the value to insert.
7. If the tree is empty:
 - Create a new node.
 - Assign it as the root.
8. Otherwise:
 - Compare the value with the current node.
 - Insert recursively into the left subtree if smaller.
 - Insert recursively into the right subtree if larger.
 - Reject duplicate values.

Case 2: Delete Node

9. If the tree is empty, display an error message.
10. Otherwise, locate the node to be deleted.
11. Perform deletion based on the node type:
 - **Leaf node:** Free the node.
 - **Node with one child:** Replace node with its child.
 - **Node with two children:**
 - Find inorder successor (minimum in right subtree).
 - Replace node value with successor value.
 - Delete the successor node.

Case 3: Search Node

12. If the tree is empty, display an error message.
13. Otherwise:
 - Compare the search value with the current node.
 - Recursively search left or right subtree.
 - Display whether the value is found or not.

Case 4: Display Tree

14. If the tree is empty, display a message.
15. Otherwise:
 - Display the tree structure rotated 90° clockwise using indentation.
 - Right subtree is displayed first, followed by root, then left subtree.

Case 5: Tree Traversals

16. If the tree is empty, display a message.
17. Otherwise, perform:
 - **Inorder Traversal (Left → Root → Right) → Sorted order**
 - **Preorder Traversal (Root → Left → Right)**
 - **Postorder Traversal (Left → Right → Root)**

Case 6: Tree Statistics

18. If the tree is empty, display a message.
19. Otherwise, compute and display:
 - Total number of nodes
 - Height of the tree
 - Minimum value (leftmost node)
 - Maximum value (rightmost node)
 - Root node value

Case 7: Exit

20. Free all allocated nodes using postorder traversal.
21. Release memory.
22. Terminate the program.

Step 7: Stop the Program

23. End execution.

OUTPUT :

```
===== MAIN MENU =====
1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit
Enter your choice: 1
Enter value to insert: 45
Inserted 45 as root node

===== MAIN MENU =====
1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit
Enter your choice: 1
Enter value to insert: 55
Inserted 55 as root node

===== MAIN MENU =====
1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit
Enter your choice: 1
Enter value to insert: 40
Inserted 40 as root node
```

```
PS D:\C programming\DHAVAMANI_EX13> gcc main.c -o main
PS D:\C programming\DHAVAMANI_EX13> ./main
===== BINARY SEARCH TREE OPERATIONS =====
```

```
===== MAIN MENU =====
```

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 4

Tree is empty!

```
===== MAIN MENU =====
```

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 1

Enter value to insert: 50

Created tree with root node 50

```
===== MAIN MENU =====
```

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 1

Enter value to insert: 47

Inserted 47 as root node

```
===== MAIN MENU =====
```

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 1

Enter value to insert: 52

Inserted 52 as root node

```
===== MAIN MENU =====
```

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 1

Enter value to insert: 57

Inserted 57 as root node

```
===== MAIN MENU =====
```

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 4

Binary Tree Structure (rotated 90° clockwise):

```
=====
```

```

                    57
                55
            52
        50
    47
    45
    40

```

===== MAIN MENU =====

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 5

--- Tree Traversals ---

1. Inorder (Sorted Order): 40 45 47 50 52 55 57
2. Preorder: 50 45 40 47 55 52 57
3. Postorder: 40 47 45 52 57 55 50

===== MAIN MENU =====

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 6

--- Tree Statistics ---

Total Nodes: 7
Tree Height: 3
Minimum Value: 40
Maximum Value: 57
Root Value: 50

===== MAIN MENU =====

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 3

Enter value to search: 52

Value 52 found in the tree

===== MAIN MENU =====

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 3

Enter value to search: 51

Value 51 not found in the tree

===== MAIN MENU =====

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 4

Binary Tree Structure (rotated 90° clockwise):

=====

```
      57
     /  \
    /    \
   50     52
  /  \   /  \
 45  47 40  47
```

===== MAIN MENU =====

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 3

Enter value to search: 51

Value 51 not found in the tree

===== MAIN MENU =====

1. Create/Insert Node
2. Delete Node
3. Search Node
4. Display Tree
5. Tree Traversals
6. Tree Statistics
7. Exit

Enter your choice: 2

Enter value to delete: 55

Deleted node with value 55 (replaced with inorder successor 57)

Deleted leaf node with value 57